

Chapter 2: Object Storage And Data Lake Layers

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

Object Storage And Data Lake Layers

Chapter 1 started with local files.

Chapter 2 moves the data path into shared object storage.

The goal is not only “upload a file”.

The goal is reproducible, scalable data management:

- data survives beyond one laptop
- teams share source locations
- layers separate raw and prepared data
- outputs can be rebuilt instead of guessed



The Data Lake Backbone

Professional AI work cannot live in:

- one laptop folder
- email attachments
- Git history
- a shared directory with mystery files

Chapter 2 introduces our shared data lake, built on object storage using an S3-style API.



The Cloud Default For Data

Object storage is the cloud-native way to keep large data.

It is used because it is:

- durable
- scalable
- accessible over HTTP APIs
- independent from one compute machine

AWS S3, Azure Blob Storage, Google Cloud Storage, and Ceph RGW all follow this idea.



Cheap Storage, Expensive Movement

Object storage is often the cheapest cloud storage for large datasets.

That makes it a good home for:

- raw history
- large assets
- model artifacts
- reproducibility evidence

But reading and moving data still costs time and money.

That is why layout, layers, and formats matter.

Compute can come and go. The durable dataset stays in the storage layer.

The Mental Shift

Stop thinking:

```
my-folder/dataset.csv
```

Start thinking:

```
s3://shared-bucket/bronze/esc50/manifest.csv
```

The location is part of the evidence.



Object Storage Is Simple

An object is:

- bytes
- a key
- metadata
- stored inside a bucket

This is very different from full filesystems.

POSIX and Windows filesystem APIs are powerful, but complex: directories, locks, permissions, in-place edits, open handles, and many edge cases.



Not A Normal Filesystem

A filesystem encourages “open, edit, save”.

Object storage encourages “write a new object”.

That creates an immutable or append-only style.

Analytics and data science usually fit this well:

- keep old inputs
- write new derived outputs
- compare versions when results change

Silent replacement is not a data strategy.

Buckets, Objects, Prefixes

Object storage does not really have folders.

It has buckets and object keys.

```
bucket: preparation-for-ai-shared  
key:    bronze/esc50/audio/dog/0001.wav
```

The slashes are naming convention.

The S3 API Is The Contract

In this course, S3 means an S3-style API.

Core operations:

- PUT: write an object
- GET: read an object
- LIST: find objects under a prefix
- HEAD: inspect metadata
- DELETE: remove when policy allows



Good Keys Are Design

A useful key answers:

- what kind of object is this?
- which project owns it?
- which source produced it?
- which version or run created it?

Names are part of your data architecture.



Storage And Compute Stay Separate

Object storage keeps the data layers stable.

Compute tools can change:

- Python
- DuckDB
- Spark
- Airflow
- BI tools
- ML training jobs

This is what makes the architecture portable.

Bronze, Silver, Gold

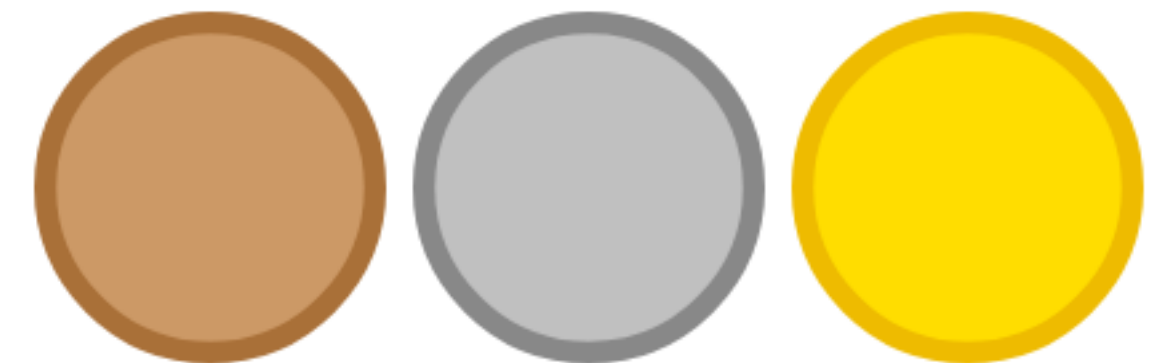
The data lake needs names for data states.

Bronze -> Silver -> Gold

Bronze asks: what did we receive?

Silver asks: what do we trust enough to analyze?

Gold asks: what can we use for a model, dashboard, or decision?



Prefixes Need Discipline

Object storage will not understand your project.

It does not know that:

- `bronze/` means raw evidence
- `silver/` means cleaned data
- `gold/` means use-case-ready data

The team gives prefixes meaning through conventions, code, and permissions.

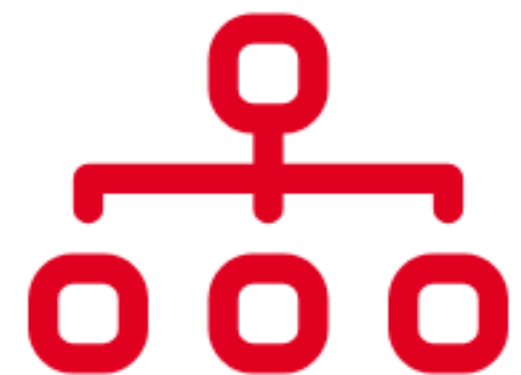
Layering Creates Traceability

A model should not be a mystery.

The team should be able to say:

```
model output  
  <- Gold feature table  
  <- Silver cleaned data  
  <- Bronze source data
```

That is the bridge from scripting to a data platform.



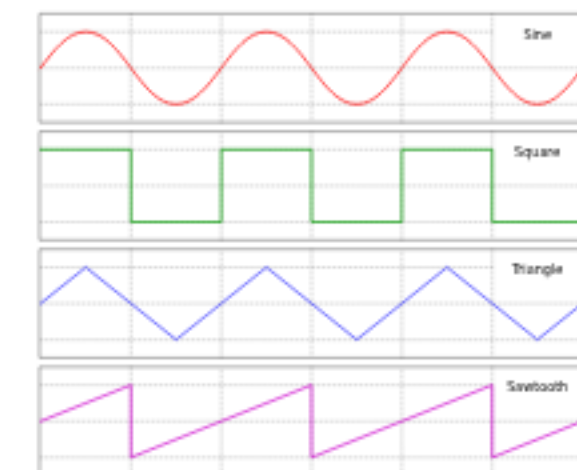
The Manifest Pattern

For assets, a table describes the files.

```
key, label, fold, bytes, sha256  
bronze/esc50/audio/dog/0001.wav, dog, 1, . . .
```

The object key tells code where the bytes are.

The manifest tells code what the bytes mean.



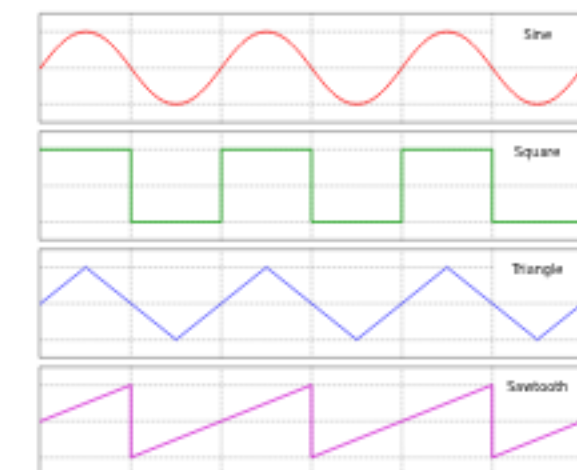
Lab Orientation

The lab builds one small data lake workflow:

shared Bronze audio

- > your Silver subset
- > your Gold feature table
- > a model trained from Gold

Every step should make bucket and key visible.



Shared Input, Personal Output

The workshop uses two bucket roles:

- shared bucket: stable read-only datasets
- participant bucket: your writable workspace

This prevents a workshop from becoming a tiny badly governed data lake.



Practical Platform Concerns

Object storage design also means thinking about:

- permissions: who can read or overwrite?
- cost: stored bytes, requests, transfer, repeated reads
- lifecycle: when should old outputs move or expire?
- versioning: is this one object version or one dataset version?

These choices become platform policy.

Object Storage And Data Lake Layers

Object storage is the data lake backbone.

Remember:

- buckets and keys replace private folders
- object storage fits append-style evidence
- conventions give prefixes meaning
- Bronze, Silver, Gold describe data states
- manifests give assets meaning
- model evidence should point back to data evidence

Next: make tabular data efficient with Parquet.